IPA Project - Phase 2

Pipelined RISC-V Processor Design

A. Pranav
2024102038
pranav.aravind@students.iiit.ac.in

Sashwat Balaji
2024102002
sashwat.balaji@students.iiit.ac.in

Sricharan Vinoth Kumar
2024112022
sricharan.v@research.iiit.ac.in

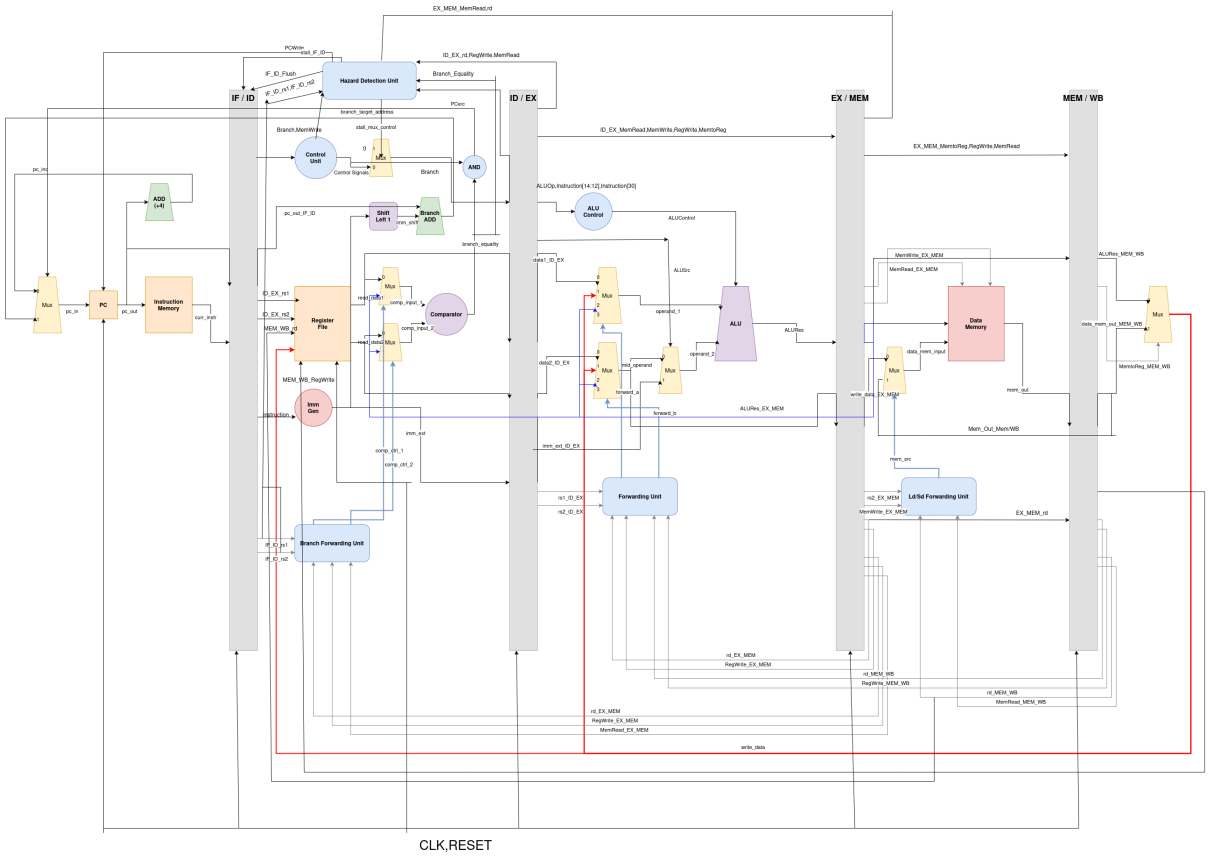

Figure 1: FINAL COMPLETE PIPE-LINED PROCESSOR The processor includes 3 forwarding units, an Hazard detection unit. We have also implemented an early branch prediction

Contents

1 Introduction **3**

2 Advanced Architectural Features (Bonus Implementations) **4**

3 Key Modifications from Sequential Architecture **5**
3.1 Negative-Edge Triggered Register File 5
3.2 Program Counter Write Enable (`PCWrite`) 5

4 Constructing the Pipelines **5**
4.1 IF/ID Pipeline Register 6
4.2 ID/EX Pipeline Register 6
4.3 EX/MEM Pipeline Register 7
4.4 MEM/WB Pipeline Register 7
4.5 Early Branch Resolution Mechanism 8

5 Detailed Pipeline Stage Diagrams **9**
5.1 Instruction Fetch (IF) Stage 9
5.2 Instruction Decode (ID) Stage 10
5.3 Execute (EX) Stage 11
5.4 Memory (MEM) Stage 12
5.5 Write Back (WB) Stage 13

6 Forwarding and Hazard Detection **13**
6.1 The Hazard Detection Unit (HDU) 14
6.2 The Forwarding Units 15
6.2.1 EX stage Forwarding Unit Logic 16
6.2.2 ld_sd Forwarding Unit 17
6.2.3 Branch_Forwarding Unit 18
6.3 Classification of Forwarding and Stalling Scenarios 18
6.3.1 Standard Data Hazards (EX-Stage Forwarding) 19
6.3.2 Load-Use Hazards (Stalls + Forwarding) 19
6.3.3 Custom Feature: Memory-to-Memory Forwarding 20
6.3.4 Custom Feature: Early Branch Resolution (ID Stage) 20

7 Simulation Results for Forwarding, Stalling and Branching cases **21**
7.1 Case 1A: EX to EX Forwarding 21
7.2 Case 1B: MEM to EX Forwarding 21
7.3 Case 1C: The Double Data Hazard 22
7.4 Case 2A: Standard Load-Use Dependency 22
7.5 Case 2B: Address Dependency for a Store 23
7.6 Case 3A: Data Dependency for a Store (ld-sd Forwarding) 24
7.7 Case 4A: Branch Dependent on an ALU Operation 24

8 Conclusion **25**

9 Team Contributions **26**

10 Acknowledgements **26**

1 Introduction

While the single-cycle sequential processor design implemented in Phase I is fully functional, it inherently suffers from severe bottlenecks regarding instruction throughput. In a sequential architecture, the processor must wait for an entire instruction to complete its execution through every single hardware module before the next instruction can even be fetched.

To overcome this limitation and extract higher performance, we implement a 5-stage Pipelined Processor. It is important to note that pipelining does not decrease the overall execution time (latency) of any single instruction. In fact, due to the setup and hold time overhead introduced by inserting pipeline registers, individual instruction latency slightly increases. However, the throughput of the processor improves dramatically.

Because the datapath is decomposed into distinct, isolated stages, a new instruction can enter the pipeline every single clock cycle. Once the pipeline is fully loaded, it creates the illusion that an entire instruction is being completed in just one clock cycle. To achieve this, the overarching datapath is separated by four pipeline registers. This separation is crucial because, in a pipelined architecture, the clock period (T_{clk}) is dictated by the slowest individual stage. By balancing the computational work across five smaller stages, we can theoretically reduce T_{clk} by a factor of 5, enabling much higher clock frequencies.

Our processor decomposes the sequential datapath into the following five distinct stages:

- **Instruction Fetch (IF):** Fetches the instruction from the Instruction Memory using the current Program Counter (PC) and increments the PC for the next cycle.
- **Instruction Decode (ID):** Decodes the fetched instruction, reads operand data from the Register File, and generates sign-extended Immediate values.
- **Execute (EX):** Utilizes the Arithmetic Logic Unit (ALU) to perform mathematical/logical operations and calculates target addresses for branching or memory access.
- **Memory (MEM):** Accesses the Data Memory module specifically to execute load and store instructions.
- **Write Back (WB):** Writes the final computed results or loaded memory data back into the Register File.

To successfully isolate these five stages, we instantiated four specific pipeline registers. Each register locks in the computed values and control signals from the previous cycle at the positive clock edge, guaranteeing stable and unchanging inputs for the subsequent stage during the current cycle:

- **IF/ID Pipeline Register:** Acts as the interface between the Instruction Fetch (IF) and Instruction Decode (ID) stages. It captures the fetched instruction and the current Program Counter, ensuring the decode logic has stable inputs to read from the Register File.
- **ID/EX Pipeline Register:** Separates the Instruction Decode (ID) and Execute (EX) stages. It latches the extracted register operands, sign-extended immediate values, and all corresponding control signals, providing stable inputs for the ALU to perform its computations.

- **EX/MEM Pipeline Register:** Forms the boundary between the Execute (EX) and Memory (MEM) stages. It locks in the computed ALU result (often a memory address) and the data intended for storage, ensuring stable inputs for Data Memory access.
- **MEM/WB Pipeline Register:** Interfaces the Memory (MEM) and Write Back (WB) stages. It captures the retrieved memory data alongside the final ALU computation, guaranteeing the correct, stable value is written back to the Register File before the instruction retires.

However, executing multiple instructions simultaneously in overlapping stages introduces structural, data, and control dependencies. When a younger instruction attempts to fetch an operand that has not yet been computed or stored by an older, executing instruction, a hazard occurs. To resolve these hazards, we implemented two primary mechanisms:

- **Data Forwarding Units:** These combinational networks bypass data directly from pipeline registers to the required inputs, allowing execution to continue without wasting clock cycles.
- **Hazard Detection Unit (HDU):** When forwarding is impossible (such as a load-use hazard where the data simply does not exist yet), the HDU stalls the pipeline to wait for the correct data to be produced.

A detailed analysis of these hazard resolution mechanisms is provided in the subsequent sections.

2 Advanced Architectural Features (Bonus Implementations)

Beyond the standard 5-stage pipeline, our processor incorporates custom architectural enhancements designed to extract maximum performance, handle complex edge cases, and minimize wasted clock cycles. These features go beyond textbook implementations to create a highly optimized datapath.

- **Early Branch Resolution (ID Stage):**

  To minimize the control hazard penalty, we migrated the branch condition evaluation logic from the Execute (EX) stage directly into the Instruction Decode (ID) stage. This modification significantly reduces the branch misprediction penalty by 1 clock cycle, meaning the hardware only ever has to flush a single instruction instead of two.

  To make this possible, we designed a custom ID-stage forwarding unit equipped with its own multiplexers. We also added specialized stall conditions to the Master Hazard Detection Unit (HDU), ensuring the ID-stage comparator always receives the correct, most up-to-date operands before triggering a flush.

- **Memory-to-Memory (Load-Store) Forwarding:**

We engineered a custom forwarding path specifically to bypass a restrictive load-use stall scenario. In a standard pipeline, a load instruction followed immediately by a dependent store always triggers a 1-cycle stall.

However, if a store instruction solely depends on a preceding load for its *data* (the `rs2` value to be stored) and not its base address (`rs1`), a stall is mathematically unnecessary. We modified the Master HDU to actively suppress the stall in this exact scenario.

Instead, the loaded data is forwarded directly from the `MEM_WB` pipeline register into the Data Memory's write port precisely when the store needs it, successfully saving a valuable clock cycle.

3 Key Modifications from Sequential Architecture

Transitioning from a single-cycle sequential datapath to a fully pipelined architecture necessitated several low-level hardware modifications to maintain data integrity and handle dynamic stalling.

3.1 Negative-Edge Triggered Register File

In the pipelined implementation, the Register File's clock sensitivity was modified to trigger on the negative edge of the clock. This design choice elegantly resolves internal structural hazards. By writing data to the registers during the first half of the clock cycle (falling edge), the fresh data can be read combinationally during the second half of the same cycle. This eliminates the need for a dedicated `WB`-to-`ID` forwarding unit, saving hardware area and ensuring that the ID-stage branch comparators have ample time to sample the correct operands before the next rising edge.

3.2 Program Counter Write Enable (`PCWrite`)

In the sequential architecture, the Program Counter (PC) updated unconditionally every clock cycle, toggling solely between `PC + 4` and a calculated branch target. In a pipelined processor, however, hazard resolution frequently requires the pipeline to stall. To achieve a stall, the state of the Instruction Fetch (IF) stage must be frozen. Therefore, a `PCWrite` control signal was introduced. Driven by the Master HDU, this signal disables the PC flip-flop, forcing it to hold its current value and repeatedly fetch the same instruction until the hazard is resolved.

4 Constructing the Pipelines

To achieve higher instruction throughput compared to the sequential processor, the datapath was decomposed into five distinct stages separated by four pipeline registers. These registers securely latch the data and control signals at the positive edge of the clock, ensuring isolated execution across different stages.

4.1 IF/ID Pipeline Register

The IF/ID pipeline register sits between the Instruction Fetch and Instruction Decode stages. Its primary purpose is to hold the fetched instruction and the current Program Counter while the decode stage processes them. It uniquely includes synchronous stall and flush controls to handle structural and control hazards early in the pipeline.

Inputs:

- `instr [31:0]`: The raw instruction fetched from instruction memory.
- `pc [63:0]`: The current Program Counter value.
- `clk, reset`: Standard clock and asynchronous reset signals.
- `flush`: Control signal to clear the register (e.g., on a taken branch).
- `stall`: Control signal to freeze the current state (e.g., on a load-use hazard).

Outputs:

- `instr_out [31:0]`: The instruction passed to the decode stage.
- `pc_out [63:0]`: The Program Counter passed to the decode stage.

4.2 ID/EX Pipeline Register

The ID/EX pipeline register buffers the boundary between the Decode and Execute stages. It carries an extensive amount of data, including the extracted source registers, immediate values, and all control signals required by the ALU, Data Memory, and Write Back stages.

Inputs:

- `rd, rs1, rs2 [4:0]`: Register file addresses for destination and sources.
- `reg_data1, reg_data2 [63:0]`: Data read from the register file.
- `imm_ext [63:0]`: The sign-extended immediate value.
- `func7, func3 [2:0]`: Function codes for ALU operation selection.
- `RegWrite, MemtoReg, ALUSrc, ALUOp [1:0], MemRead, MemWrite`: Control signals.
- `clk, reset`: Standard clock and reset signals.

Outputs:

- `rd_out, rs1_out, rs2_out [4:0]`: Register addresses passed to Execute.
- `reg_data1_out, reg_data2_out [63:0]`: Register data passed to Execute.
- `imm_ext_out [63:0]`: Extended immediate passed to Execute.
- `func7_out, func3_out [2:0]`: Function codes passed to Execute.
- `RegWrite_out, MemtoReg_out, ALUSrc_out, ALUOp_out [1:0], MemRead_out, MemWrite_out` Control signals propagated forward.

4.3 EX/MEM Pipeline Register

The EX/MEM pipeline register acts as the buffer before the Data Memory stage. It stores the computed ALU result (which often serves as a memory address) and the data intended to be stored. Notably, it also carries the `rs2` address forward to enable advanced memory-to-memory data forwarding.This forwarding from `MEM_WB` to `MEM` stage is something we have implemented too, which we will explain in the upcoming sections.

Inputs:

- `rd, rs2 [4:0]`: Destination register and the source 2 register (carried for forwarding logic).
- `ALURes [63:0]`: The 64-bit result calculated by the ALU.
- `Write_Data [63:0]`: The data meant to be stored in memory (rs2 data).
- `MemWrite, MemRead, RegWrite, MemtoReg`: Control signals for Memory and Write Back.
- `clk, reset`: Standard clock and reset signals.

Outputs:

- `rd_out, rs2_out [4:0]`: Register addresses passed to Memory stage.
- `ALURes_out [63:0]`: ALU result passed as memory address or to Write Back.
- `Write_Data_out [63:0]`: Data sent to Data Memory's write port.
- `MemWrite_out, MemRead_out, RegWrite_out, MemtoReg_out`: Control signals propagated forward.

4.4 MEM/WB Pipeline Register

The MEM/WB pipeline register is the final buffer before the instruction retires. It holds the data retrieved from memory alongside the data calculated by the ALU, allowing the Write Back stage to multiplex the correct value into the Register File.

Even in this `MEM_WB` pipeline register, we have carried forward the `MemRead` and `RegWrite` signals. This is to check if the instruction currently in the `WB` stage is a load instruction; if it is, we then check the instruction in the `EX_MEM` pipeline register. If the destination register of the load matches the `rs2` register of a store instruction, we need to perform `ld-sd` (Memory-to-Memory) forwarding. This is why we have carried these control signals into the `MEM_WB` pipeline register as well.

Inputs:

- `rd [4:0]`: The destination register address.
- `Data_mem [63:0]`: The 64-bit data loaded from Data Memory.
- `ALURes [63:0]`: The 64-bit result calculated previously by the ALU.
- `RegWrite, MemtoReg, MemRead`: Control signals for Write Back and forwarding unit checks.

- `clk, reset`: Standard clock and reset signals.

Outputs:

- `rd_out [4:0]`: Destination register address passed to Register File.
- `Data_mem_out [63:0]`: Memory data passed to Write Back multiplexer.
- `ALURes_out [63:0]`: ALU result passed to Write Back multiplexer.
- `RegWrite_out, MemtoReg_out, MemRead_out`: Control signals driving Write Back and forwarding.

4.5 Early Branch Resolution Mechanism

To optimize our pipeline and reduce the penalty of control hazards, the branch evaluation logic was relocated from the Execute (EX) stage directly into the Instruction Decode (ID) stage. This architectural change involves several key hardware modifications .

Hardware Relocation and Additions:

- **Address Calculation:** The Shift Left Logical (SLL) module and the branch target adder were moved into the ID stage to compute the target address immediately after decoding the instruction.
- **Dedicated Comparator:** Because the ID stage lacks an ALU to perform subtraction and check the zero flag, a dedicated equality comparator was introduced. It compares the two operand values and asserts a `branch_equalizer` signal if they are equal.

  The dedicated comparator operates by performing a bitwise XOR on every individual bit of the two 64-bit operands. The resulting 64 bits are then passed through a wide logical OR operation. If the output of this OR is a 1, it implies that at least one bit differed, meaning the two operands are not equal. Therefore, we simply negate (NOT) the final OR output to generate the active-high `branch_equalizer` signal.
- **Forwarding Multiplexers:** Two 2-to-1 Multiplexers (MUXes) were placed directly in front of the comparator. Controlled by our new Branch Forwarding Unit, these MUXes select the correct operand data—either directly from the Register File reads or forwarded from the `EX_MEM` pipeline's `ALURes`.

Flush and Stall Logic Integration: Evaluating branches early requires strict coordination with the Hazard Detection Unit (HDU) to ensure data integrity:

- **Operand Readiness:** If a branch instruction depends on an operand still being computed in the EX or MEM stages, the processor must stall. The forwarding unit and HDU handle this by asserting the `master_stall` signal, effectively making the comparator wait until the correct values arrive.
- **Triggering the Flush:** The `IF_ID` pipeline register is only flushed when three exact conditions are met simultaneously: the instruction is a branch (`branch == 1`), the operands are equal (`branch_equalizer == 1`), AND the processor is not stalling (`master_stall == 0`). Ensuring the stall has ended guarantees that the branch decision is based on the correct, fully forwarded data.

The Architectural Trade-off: By resolving the branch in the ID stage, a branch misprediction requires flushing only one instruction (the one currently in the IF stage). While this drastically reduces the flush penalty, we accept the trade-off that acquiring the correct data for the ID-stage comparison might occasionally require stalling the pipeline to wait for an executing instruction to finish.

5 Detailed Pipeline Stage Diagrams

5.1 Instruction Fetch (IF) Stage

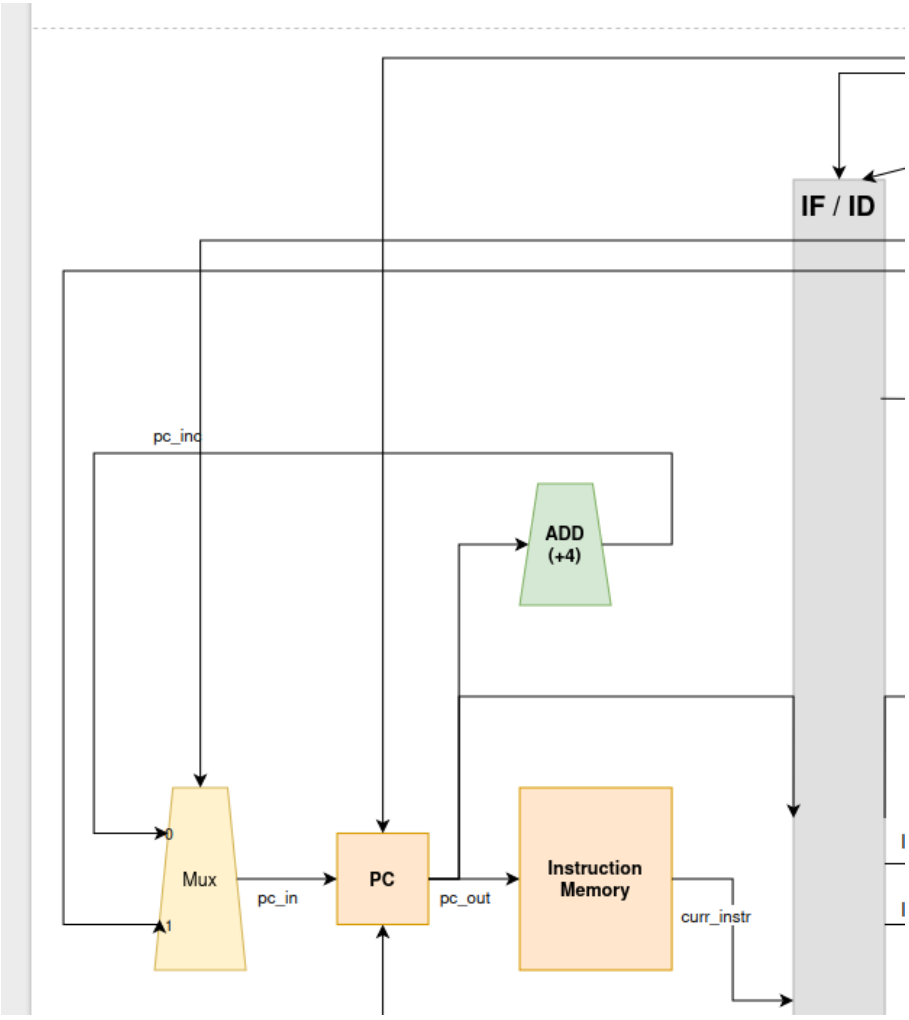

Figure 2: Instruction Fetch (IF) Stage datapath and connections

5.2 Instruction Decode (ID) Stage

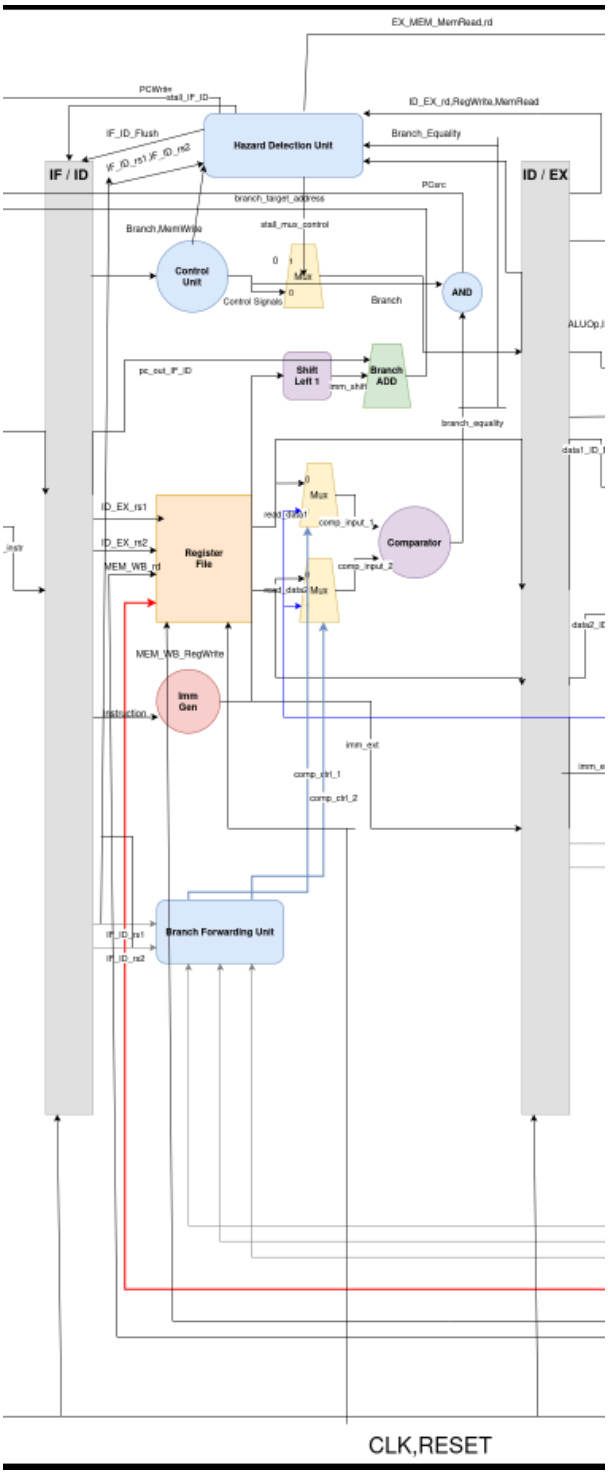

Figure 3: Instruction Decode (ID) Stage featuring the Early Branch Forwarding Unit and Dedicated Comparator.

5.3 Execute (EX) Stage

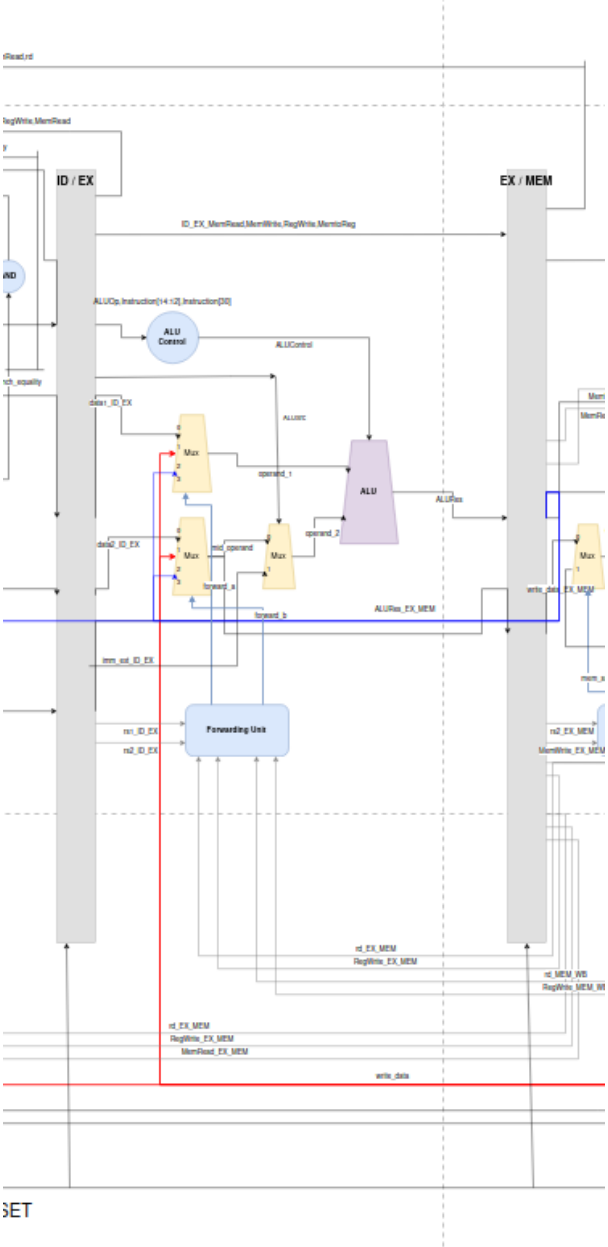

Figure 4: Execute (EX) Stage featuring the ALU and the Standard Forwarding Unit.

5.4 Memory (MEM) Stage

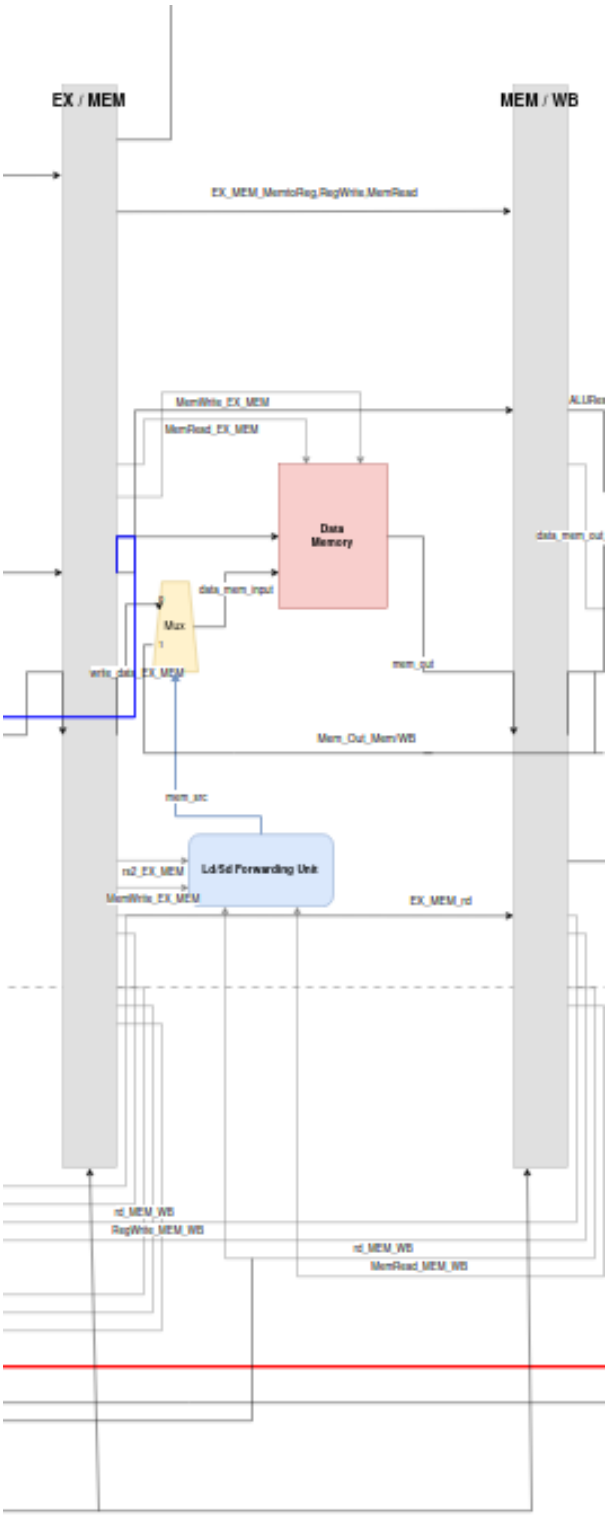

Figure 5: Memory (MEM) Stage featuring the Data Memory and the Custom Load-Store Forwarding Unit.

5.5 Write Back (WB) Stage

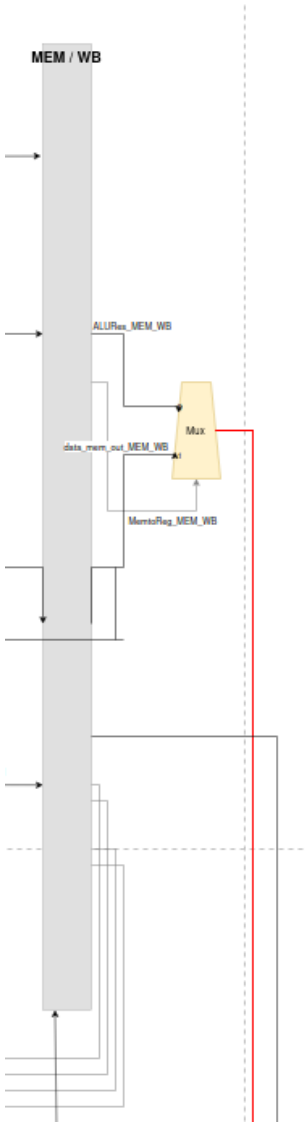

Figure 6: WriteBack (WB) Stage

6 Forwarding and Hazard Detection

This section details the central control logic of our design, which effectively acts as the brain and nervous system of the pipelined processor. Without this crucial logic, the processor would simply fetch the next instruction and blindly read operands from the Register File, completely unaware of whether it is fetching the correct, most recently updated values.

Identifying and intercepting such data hazards is absolutely critical for the correct execution of any given sequence of instructions. This necessity dictates the implementation of our Hazard Detection Unit (HDU) and Forwarding Units. In many ways, these two hardware components go hand-in-hand to maintain both data integrity and optimal throughput.

6.1 The Hazard Detection Unit (HDU)

The primary purpose of the HDU is to identify impending data threats and stall the pipeline whenever it detects a data hazard taking place. If the HDU sees that an instruction is trying to fetch an operand whose latest value has not yet been computed or loaded, it issues a set of control signals that tells the processor to stop fetching new instructions. By freezing the pipeline stages before the hazard, the HDU protects the processor from consuming stale, incorrect data.

```
`timescale 1ns/1ns

module hazard_detection_unit(
    input [4:0] IF_ID_rs1,
    input [4:0] IF_ID_rs2,
    input Branch,
    input MemWrite,
    input Equal,
    input [4:0] ID_EX_rd,
    input ID_EX_MemRead,
    input ID_EX_RegWrite,
    input [4:0] EX_MEM_rd,
    input EX_MEM_MemRead,
    output PCWrite,
    output IF_ID_Write,
    output Stall_Flush,
    output IF_Flush
);

    wire load_use_stall = ID_EX_MemRead && (ID_EX_rd != 5'b0) && ( (ID_EX_rd == IF_ID_rs1) || ((ID_EX_  rd == IF_ID_rs2) && ~MemWrite) );
    wire branch_stall_EX = Branch && ID_EX_RegWrite && (ID_EX_rd != 5'b0) && ((ID_EX_rd == IF_ID_rs1) || (ID_EX_rd == IF_ID_rs2));
    wire branch_stall_MEM = Branch && EX_MEM_MemRead && (EX_MEM_rd != 5'b0) && ((EX_MEM_rd == IF_ID_rs1) || (EX_MEM_rd == IF_ID_rs2));
    wire master_stall = load_use_stall | branch_stall_EX | branch_stall_MEM;

    assign PCWrite     = ~master_stall;
    assign IF_ID_Write = ~master_stall;
    assign Stall_Flush = master_stall;
    assign IF_Flush    = Branch & Equal & ~master_stall;

endmodule
```

Figure 7: HDU Logic Code

Logic Explanation:

Honestly, we found this to be the most challenging and fun aspect of this project. The HDU deals with the cases where stalling is essential for either early branch prediction or when we need to forward data to the EX stage.

So how do we logically determine if for an instruction present in the ID stage stalling is essential or not?

- **Case 1:** Say you have a branch instruction in the ID stage, this implies that I must have the right operands now. If not, I am going to stall until my forwarding unit provides me with the right data. The HDU is adamant in that way.
  - If we have `rd_ID_EX` matching with either `rs1_IF_ID` or `rs2_IF_ID`, we need to stall irrespective of what the instruction in the `ID_EX` pipeline register is.
  - This is because no computation has happened yet when the data is locked into the `ID_EX` stage.
  - Hence, any forwarding can happen only after the instruction (the one to be forwarded to ID) moves to the `EX_MEM` stage. Therefore, we require a stall here.
- **Case 2:** Say we again have a branch instruction in the ID stage, but now the match of `rd` with either `rs1` or `rs2` happens from the `EX_MEM` stage.

- i.e., `rd_EX_MEM` matches with either `rs1_IF_ID` or `rs2_IF_ID`. In this case, I will stall only if the instruction is a load type instruction.
- This is because if it was an R-type normal instruction, I would have already forwarded the data from the `ALURes_EX_MEM` to the MUX present in the ID stage.
- Hence if I stall again, it would be a waste of a cycle.
- However, if the match happened and the instruction present in the `EX_MEM` pipeline is a `ld` type instruction, then we stall again since the instruction hasn't yet accessed the memory.
- This is exactly the logic we have implemented for `branch_stall_MEM`. *(NOTE: All these stallings happen only after verifying that the `rd` is non `x0`.)*

- **Case 3: The load-use stall.** This is not exactly the conventional `ld`-use stall since we have added a refinement to this for the purpose of the `ld-sd` forwarding.
  - We will not stall even if we have a `ld` in the EX stage and an instruction with the `rs2_IF_ID` equaling the `rd_EX_MEM` if the instruction in the ID stage is a store.
  - This is because we do not require the loaded data in the EX stage, it is only the address that gets calculated there.
  - Hence it is enough if we have the `Write_Data` by the time we reach the `MEM` stage, where the `ld-sd` forwarding unit takes care of the same.
  - However, we should stall if there is a store instruction but the address calculation itself needs the loaded data's value! This is why the logic reads:

    ```
    wire load_use_stall = ID_EX_MemRead && (ID_EX_rd != 5'b0) &&
        ( (ID_EX_rd == IF_ID_rs1) || ((ID_EX_rd == IF_ID_rs2) && ~MemWrite) );
    ```

  - This logic tells us that we stall in Case 3 only if the `ID_EX_MemRead` is high, meaning it is a load instruction (writes into non `x0`).
  - And either the `rs1_IF_ID` matches with the `rd_ID_EX`, or the `rs2` matches but with a `~MemWrite`.
  - The reason the last phrase is present is to indicate that in case of a store instruction here, we have an exception for `rs2` by letting it equal the `rd`, but still we do not stall since our forwarding unit takes care of the same!

The Case 3 logic is what we found the most conceptual and interesting to come up with!

6.2 The Forwarding Units

While the HDU applies the stalls, the Forwarding Units are what ultimately stop the stalls. Because the HDU stalled the pipeline waiting for a new value to be computed, the Forwarding Unit's job is to physically bypass the standard datapath and route that newly computed data directly to where it is needed, bypassing the Register File entirely.

To handle all potential data hazards efficiently, we have implemented four distinct forwarding paths:

- **`EX_MEM` to `EX` Stage:** Forwards the most recent ALU result directly to the next instruction's ALU input, resolving standard consecutive data dependencies.
- **`MEM_WB` to `EX` Stage:** Forwards data that is about to be written back into the Register File to the ALU input, handling delayed dependencies.
- **`MEM_WB` to `MEM` Stage (`ld`-`sd` Forwarding):** A custom optimization for load-store sequences. If the store instruction only depends on the load for its data (and has no dependency on `rs1` for address calculation), the loaded data is forwarded directly to the Data Memory write port, requiring zero stalls.
- **`EX_MEM` to `ID` Stage (Early Branch Prediction):** Supports our advanced early branch resolution by forwarding ALU results directly to the newly relocated ID-stage comparators, ensuring the branch logic has the correct operands without waiting for the Write Back stage.

6.2.1 EX stage Forwarding Unit Logic

```
module forwarding_unit(ex_mem_rd,ex_mem_regwrite,mem_wb_rd,mem_wb_regwrite,id_ex_rs1,id_ex_rs2,forward_a,forward_b);

    input wire [4:0] ex_mem_rd,mem_wb_rd,id_ex_rs1,id_ex_rs2;
    input wire ex_mem_regwrite,mem_wb_regwrite;
    output reg [1:0] forward_a,forward_b;

    always @(*) begin

        forward_a=2'b00;
        forward_b=2'b00;

        if(ex_mem_rd!=5'b0 && ex_mem_regwrite==1'b1 && id_ex_rs1==ex_mem_rd) begin
            forward_a=2'b10;
        end

        else if(mem_wb_rd!=5'b0 && mem_wb_regwrite==1'b1 && id_ex_rs1==mem_wb_rd) begin
            forward_a=2'b01;
        end

        if(ex_mem_rd!=5'b0 && ex_mem_regwrite==1'b1 && id_ex_rs2==ex_mem_rd)begin
            forward_b=2'b10;
        end
        else if(mem_wb_rd!=5'b0 && mem_wb_regwrite==1'b1 && id_ex_rs2==mem_wb_rd) begin
            forward_b=2'b01;
        end

    end
endmodule
```

Figure 8: EX stage forwarding unit

Logic Explanation:

This forwarding unit is to forward data from the `EX_MEM` pipeline register or from the `MEM_WB` pipeline register to the EX stage.

This forwarded data could go to either operand1 of the ALU or the operand2, hence we have identical cases for both (to compare).

We have 2 MUXs present at the output of the `ID_EX` pipeline registers, these are 4x1 MUXs. We will use 3 of these inputs for our purpose.

- Initially we set the select inputs of these MUXs i.e. `forward_a` and `forward_b` to 0. Now we check for dependency.

- If we find that in the EX_MEM pipeline, there is an instruction whose rd matches with either rs1 or rs2 and it writes back to a register address that is not x0, then we set the forward control of that MUX to 10.

- forward = 10 indicates that the forwarding happens from the EX_MEM pipeline to the EX stage.

- The same logic is used for the forwarding from MEM_WB to the EX stage but now the rd and reg_write of the MEM_WB pipeline is used for the comparison. In this case the forward is set to 01.

- forward = 01 indicates that the forwarding happens from the MEM_WB pipeline to the EX stage.

6.2.2 ld_sd Forwarding Unit

```
`timescale 1ns/1ns

module ld_sd_fwd(
    input [4:0] EX_MEM_rs2,
    input EX_MEM_MemWrite,
    input [4:0] MEM_WB_rd,
    input MEM_WB_MemRead,
    output Forward_Mem
);
assign Forward_Mem = EX_MEM_MemWrite & MEM_WB_MemRead & (MEM_WB_rd != 5'b0) & (MEM_WB_rd == EX_MEM_rs2);

endmodule
```

Figure 9: ld_sd Forwarding Logic

Logic Explanation:

If without stalling a load followed by a store instruction has reached the MEM_WB and the EX_MEM stage, then there is either no dependency on each other, or there is the only possibility of the value being stored depends on the loaded data.

So here the case being handled is when a load is followed by a store and the data store writes into the memory is exactly what was loaded.

In this case our normal forwarding unit would have stalled without entering the EX stage. However, due to our HDU seeing that there is no dependency on the rs1 lets the sd into the EX stage.

So we aim here to detect such a ld followed by store with the write_data dependency.

- The logic says that a control signal called the Forward_Mem is made high only when the instruction in the MEM_WB is a load (checked using MEM_WB_MemRead), the rd written into is non-x0, and the instruction in EX_MEM is a sd (checked using EX_MEM_MemWrite).

- This Forward_Mem is the control input to a MUX placed right in front of the EX_MEM pipeline register.

- Its inputs are Write_Data_Out coming from the EX_MEM pipeline register and the other input comes from the output of the MUX present for the Write_back data. That is how the forwarding works in this case.

6.2.3 Branch_Forwarding Unit

```
`timescale 1ns/1ns

module Branch_Forwarding_Unit(
    input [4:0] IF_ID_rs1,
    input [4:0] IF_ID_rs2,
    input [4:0] EX_MEM_rd,
    input EX_MEM_RegWrite,
    input EX_MEM_MemRead,
    output ForwardA_ID,
    output ForwardB_ID
);

    assign ForwardA_ID = EX_MEM_RegWrite & (~EX_MEM_MemRead) & (EX_MEM_rd != 5'b0) & (EX_MEM_rd == IF_ID_rs1);
    assign ForwardB_ID = EX_MEM_RegWrite & (~EX_MEM_MemRead) & (EX_MEM_rd != 5'b0) & (EX_MEM_rd == IF_ID_rs2);

endmodule
```

Figure 10: Branch_Forwarding Logic

Logic Explanation:

The only forwarding that happens here is from the `ALURes_EX_MEM` to the input of the MUXs placed in front of the comparators.

So we might naturally have a doubt as to what about the forwarding of a load instruction? This is where the beauty of us making the register file negative edge triggered comes into play. We simply do not need to forward in case of a load operation followed by a branch at all.

- This is because, once your loaded data reaches the `MEM_WB` register, in the negative clock edge of the same cycle it is also written into the register itself, hence our `read_Data1` and `read_Data2` would actually have the right values already for the branch prediction. Hence just stalling fixes the forwarding in case of a load instruction.
- However for an R type instruction we need forwarding from the `ALURes_EX_MEM`. Also note how the Double Data Hazard is casually fixed since `forward_ID` is made high always when there is a `rd` match in the `rd_EX_MEM` stage, so we always forward from there if there is a match and provided it is not a load type instruction.
- `forwardA_ID` and `forwardB_ID` are the control signals generated by the forwarding unit for the MUXs selecting the comparator's inputs.
- So whenever the instruction in the `EX_MEM` pipeline is writing back and is not a load instruction and given the `rd_EX_MEM` is non `x0` and the `rd_EX_MEM` matches with `rs1_IF_ID` or `rs2_IF_ID` of the ID stage, we set the `forward_ID` high for that MUX.

6.3 Classification of Forwarding and Stalling Scenarios

To ensure the pipeline handles all data and control dependencies correctly without sacrificing unnecessary clock cycles, the architecture addresses several distinct categories of hazards. Below is a comprehensive classification of every stalling and forwarding scenario handled by this processor.

6.3.1 Standard Data Hazards (EX-Stage Forwarding)

These hazards occur when an instruction requires an operand that was computed by a recent ALU operation but has not yet been written back to the Register File. They are resolved purely through data routing, requiring zero stalls.

- **Case 1A: EX to EX Forwarding**
  An R-type instruction is followed immediately by a dependent instruction.

  ```
  add x1, x2, x3
  sub x4, x1, x5
  ```

  Resolution: The `sub` instruction requires `x1` in the EX stage while the `add` instruction is in the EX/MEM pipeline register. The Forwarding Unit detects the match and routes the ALU result directly to the ALU input (`ForwardA_EX = 2'b10`).

- **Case 1B: MEM to EX Forwarding**
  An R-type instruction is followed by an independent instruction, and then a dependent instruction.

  ```
  add x1, x2, x3
  nop
  sub x4, x1, x5
  ```

  Resolution: The `sub` instruction requires `x1` while the `add` instruction is in the MEM/WB pipeline register. The Forwarding Unit routes the data from MEM/WB to the ALU input (`ForwardA_EX = 2'b01`).

- **Case 1C: The Double Data Hazard**
  Two consecutive instructions write to the same register, followed by an instruction that reads it.

  ```
  addi x1, x0, 10
  addi x1, x0, 20
  add  x2, x1, x0
  ```

  Resolution: The `add` instruction needs the most recent value of `x1` (20, not 10). The hardware prioritizes the younger data, triggering the EX-to-EX forwarding path over the MEM-to-EX path.

6.3.2 Load-Use Hazards (Stalls + Forwarding)

These hazards occur when an instruction requires data that is currently being fetched from Data Memory. Because memory access takes time, a pipeline stall is strictly required.

- **Case 2A: Standard Load-Use Dependency**
  A load is followed immediately by an ALU operation requiring the loaded data.

  ```
  ld  x1, 0(x2)
  add x3, x1, x4
  ```

Resolution: The Master Hazard Detection Unit (HDU) detects the `MemRead` signal in the EX stage and freezes the IF and ID stages for 1 cycle. In the following cycle, the data is forwarded from the MEM/WB register to the EX stage.

- **Case 2B: Address Dependency for a Store**
  A load is followed by a store that uses the loaded data as its base memory address.

  ```
  ld x1, 0(x2)
  sd x3, 0(x1)
  ```

  Resolution: Because the ALU requires `x1` in the EX stage to compute the target address (`x1 + 0`), the HDU enforces a 1-cycle stall exactly as it does for a standard Load-Use hazard.

6.3.3 Custom Feature: Memory-to-Memory Forwarding

We have implemented an optimization for cases as listed below. We see in this case that the address calculation has no dependency on the value the load is going to store back; hence, we need not stall and let the `sd` instruction into the execute stage. This way, the `ld` is fetching the data in the MEM stage while the `sd` computes the destination address.

Hence, when this clock cycle ends, we forward the data to be stored directly from the MEM-WB register to the MEM stage. This works because we need the data to be stored only at the MEM stage. This will help us save 1 clock cycle by not stalling. We remember to include this case in the HDU and program it to not stall for the same.

- **Case 3A: Data Dependency for a Store**

  ```
  ld x1, 0(x2)
  sd x1, 0(x3)
  ```

  Resolution: The HDU detects that the dependency is purely on `rs2` and the consuming instruction is a Store (`MemWrite = 1`). It actively suppresses the stall. When the instructions align in the MEM and WB stages, a custom multiplexer forwards the loaded data directly from the MEM/WB register into the Data Memory's write port.

6.3.4 Custom Feature: Early Branch Resolution (ID Stage)

To minimize the control hazard penalty, branch condition evaluation is moved from the EX stage to the ID stage. This requires highly specialized forwarding and stalling logic.

- **Case 4A: Branch Dependent on an ALU Operation**

  ```
  add x1, x2, x3
  beq x1, x4, target
  ```

  Resolution: The Master HDU detects that the branch needs `x1`, which is currently being computed in the EX stage. It stalls the pipeline for 1 cycle. In the following cycle, the custom ID-Stage Forwarding Unit routes the computed result from the EX/MEM pipeline register directly into the branch equality comparator.

7 Simulation Results for Forwarding, Stalling and Branching cases

7.1 Case 1A: EX to EX Forwarding

```
add x1, x2, x3
sub x4, x1, x5
```

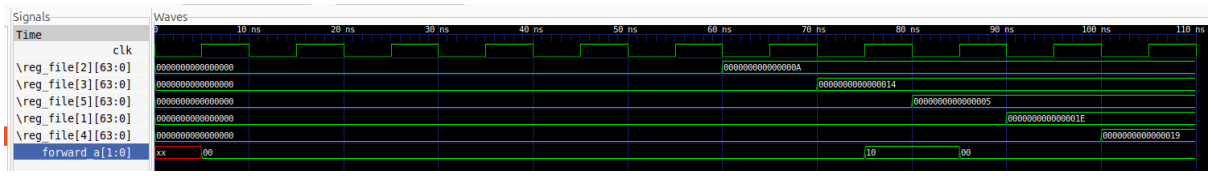

Figure 11: EX to EX Forwarding

Explanation:

- We see the `forward_A` signal being set to 10.
- This indicates that forwarding is taking place from the `EX_MEM` pipeline register to the ALU's first operand.
- We also see that the resulting dependent subtraction is written right after the `add` itself gets written into the Register File.
- Since no stalling has been utilized, the EX-to-EX forwarding is fully functional.

7.2 Case 1B: MEM to EX Forwarding

```
add x1, x2, x3
nop
sub x4, x1, x5
```

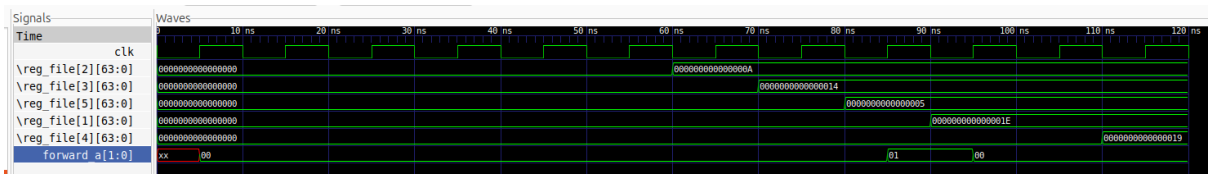

Figure 12: Simulation waveform demonstrating MEM to EX Forwarding.

Explanation:

- We notice in the waveform that `x4` is written into the Register File one clock cycle after the writing of `x1`, which is expected due to the `nop` in between.
- When the `add` instruction is in the `MEM_WB` pipeline, the `sub` instruction has already reached the ALU.

- However, it had fetched stale data because the `add`'s result is only just being written back into the Register File.
- Thus, we see forwarding occur. A `forward_A = 01` signal indicates the forwarding of the `MEM_WB` Writeback MUX's data to the ALU's first operand.
- Hence, the forwarding from the `MEM_WB` to the EX stage is also functional.

7.3 Case 1C: The Double Data Hazard

```
addi x1, x0, 10
addi x1, x0, 20
add  x2, x1, x0
```

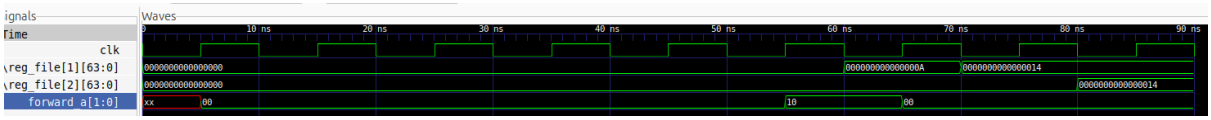

Figure 13: Simulation waveform demonstrating the resolution of a Double Data Hazard.

Explanation:

- Notice how `x1`'s value is first made 10, then changed to 20 just before the `add` instruction needs it as an operand.
- In this case, we have `x1 = 10` present in the `MEM_WB` pipeline register and `x1 = 20` present in the `EX_MEM` pipeline register.
- This creates two matches for the `rd` with respect to the input operands (`rs1`/`rs2`).
- However, we must forward the data that is the newest, most recently updated value.
- For this reason, the forwarding happens from the `EX_MEM` pipeline register, as indicated by `forward_A = 10` in the waveform.

7.4 Case 2A: Standard Load-Use Dependency

```
ld  x1, 0(x2)
add x3, x1, x4
```

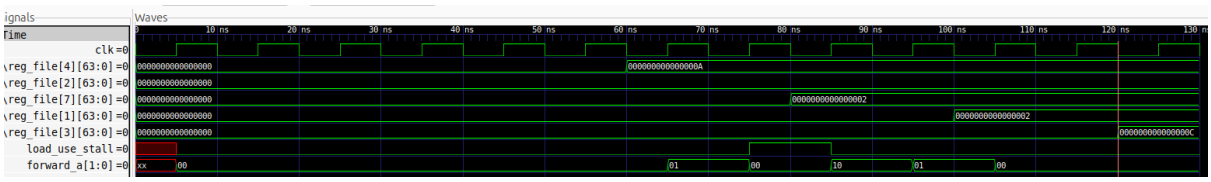

Figure 14: Simulation waveform demonstrating a pipeline stall for a standard Load-Use dependency.

Explanation:

- Notice that even though the `ld` and `add` instructions are back-to-back, they arrive with a clock cycle gap when hitting the Register File, indicating a stall took place.
- When the `ld` instruction moves from the EX stage to the MEM stage, the `load_use_stall` signal goes high.
- This tells the IF and ID stages to stall while the `ld` moves into the memory stage.
- There is only one stall cycle, as the loaded data is ready at the end of the stalling clock edge.
- The `forward_A` signal becomes `01` for the clock edge immediately after stalling, indicating data is forwarded from the `MEM_WB` pipeline register to the EX stage.
- Hence, the load-use stall is functional.

7.5 Case 2B: Address Dependency for a Store

```
ld x1, 0(x2)
sd x3, 0(x1)
```

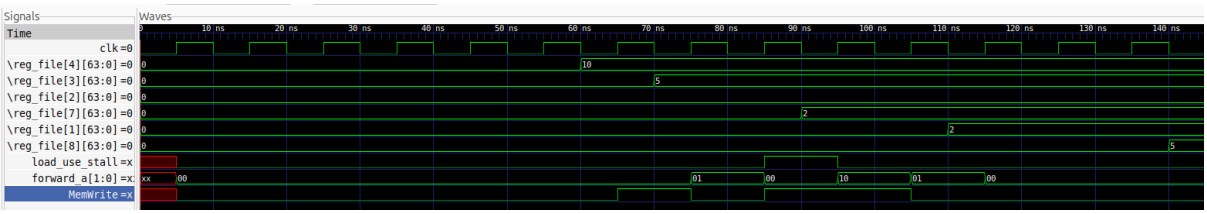

Figure 15: Simulation waveform demonstrating a stall for an address calculation dependency.

Explanation:

- Note how we are stalling even though this is a load followed by a store instruction.
- While we built the `ld-sd` forwarding unit specifically to avoid stalling, it only works when the address calculation is independent of the loaded data.
- Only when the address calculation is independent can we let the `sd` instruction into the EX stage and directly feed the data to be stored in the MEM stage.
- Here, the address calculation itself requires the loaded data as an input, so we must stall when the `rd` matches `rs1`.
- During the stall, the `MemWrite` signal is extended because the `sd` is stalled in the ID stage while the `ld` moves to the MEM stage (an `nop` is inserted into the EX stage).
- The `01` signal for `forward_A` occurs when the `sd` finally enters the EX stage, directly providing the base address from `x2` to the ALU operand.
- As seen next, if the match is with `rs2`, we only need to stall for non-`sd` instructions.

7.6 Case 3A: Data Dependency for a Store (ld-sd Forwarding)

```
ld x1, 0(x2)
sd x1, 0(x3)
```

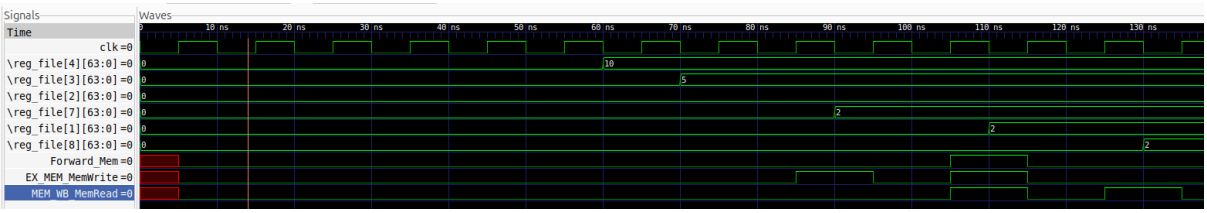

Figure 16: Simulation waveform demonstrating the custom Memory-to-Memory forwarding without stalling.

Explanation:

- Notice that we do not stall in this case because the dependency is on `rs2` and it is an `sd` instruction following an `ld` instruction.
- Therefore, the HDU does not stall the pipeline.
- At 105ps, the `Forward_Mem`, `EX_MEM_MemWrite`, and `MEM_WB_MemRead` signals go high.
- This indicates that the instruction in `MEM_WB` is a load, the one in `EX_MEM` is a store, and the `rd` of `MEM_WB` matches the `rs2` of `EX_MEM`.
- Consequently, we forward the data from the `MEM_WB` pipeline to the MUX in front of the Data Memory.
- We can confirm this by observing the `Forward_Mem` signal going high, indicating forwarding from the next pipeline stage.

7.7 Case 4A: Branch Dependent on an ALU Operation

```
add x1, x2, x3
beq x1, x4, target
```

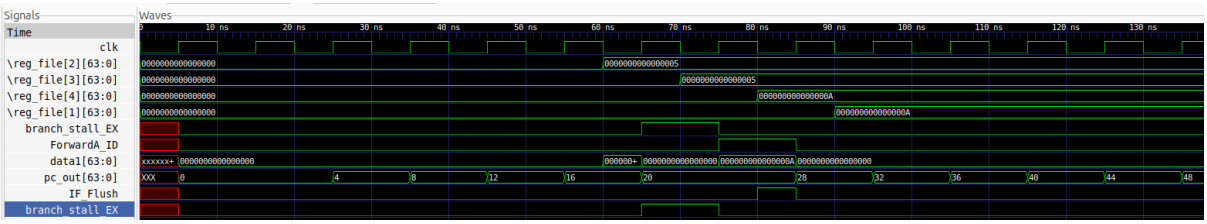

Figure 17: Simulation waveform demonstrating early branch resolution in the ID stage.

Explanation:

- First, when the `add` is in the EX stage and the `beq` is in the ID stage, the stall is set high because the `beq`'s operands are not yet computed by the ALU until the end of this clock cycle.

- Notice how the forwarding takes place during the stalling cycle.
- `ForwardA_ID = 1` indicates that instead of the Register File's contents, we choose the `ALURes_EX_MEM` as the input to the comparator.
- This happens because it is the most recent value, and there is a match between `rd_EX_MEM` and `rs1_ID_EX`.
- This naturally resolves the double data hazard, as we always forward upon a match in the `EX_MEM` stage since it holds the newest information.
- In our case, the branch is taken, and we see the `IF_flush` signal going high.
- This indicates that at the end of the cycle where `ForwardA_ID` goes high, the decision is made to flush the IF stage's data and execute a PC jump.
- We can see the PC go directly from 20 to 28, confirming exactly where the branch occurred.

8 Conclusion

Moving from our Phase I sequential design to a fully pipelined 5-stage RISC-V processor was a significant jump in complexity, but seeing the instruction throughput improve made the effort worthwhile. Designing the Hazard Detection Unit (HDU) and our three distinct Forwarding Units was arguably the most challenging and engaging aspect of the entire project. Figuring out exactly when to stall the pipeline and when to cleverly bypass it with forwarded data felt like solving a massive puzzle.

Rather than sticking strictly to the baseline textbook implementation, we focused on custom refinements to optimize the datapath. By shifting branch evaluation to the ID stage and using a negative-edge triggered register file, we efficiently mitigated control hazards. Additionally, our custom `ld-sd` forwarding unit actively suppresses the conventional load-use stall, saving valuable clock cycles.

Ultimately, our processor successfully executes complex instruction sequences while maintaining strict data integrity. Building this architecture and implementing these specific features gave us a profound, hands-on understanding of what really happens under the hood of modern processors.

Designing the Hazard Detection Unit (HDU) and the three distinct Forwarding Units was undoubtedly the most challenging, yet conceptually rewarding, aspect of this endeavor. By moving beyond baseline textbook implementations—specifically by engineering Early Branch Resolution in the ID stage and a custom Memory-to-Memory (`ld-sd`) forwarding path—we effectively minimized wasted clock cycles and mitigated severe control and data hazards.

The processor now successfully executes complex instruction sequences, intelligently prioritizing younger data, actively suppressing unnecessary pipeline stalls, and maintaining strict data integrity. Ultimately, this project provided profound, hands-on insight into the intricate hardware-software interface and the meticulous logic required to maximize computational throughput.

9 Team Contributions

This project was the result of a collective, intensive effort by all three team members. We were involved in synchronized brainstorming and live-coding sessions to architect the datapath and debug the control logic.

Because every design decision—from the initial pipeline decomposition to the complex implementation of the three forwarding units and the Hazard Detection Unit—was conceived and coded together as a team, we have chosen not to compartmentalize individual tasks. Every line of Verilog and every architectural optimization reflects our combined effort and shared technical intuition.

10 Acknowledgements

We would like to express our sincere gratitude to Professor Dr. Priyesh Shukla for providing us with the opportunity to undertake this project. We also extend our thanks to the Teaching Assistants for their support and guidance .

Resources

[1] David A. Patterson and John L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface, RISC-V Edition*. Morgan Kaufmann, 2018.